B27 Serving and monitoring models

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2

B27 Serving and monitoring models 3

B27-L1 The model registry, versions and aliases 3

B27-L2 Serving a model with FastAPI 8

B27-L3 Packaging a model service as a container image 14

B27-L4 Deploying the container to Azure Container Apps 19

B27-L5 Monitoring a served model for drift 25

B27-L6 Retraining, promotion and rollback 30

Track B best-practice checklist 37

Appendix: solutions and answer keys 38

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B27 Serving and monitoring models

Roadmap: Months 16 to 18: MLOps and deployment. The model registry and aliases, a FastAPI endpoint, container images, Azure Container Apps, drift monitoring, and retraining with rollback.

B27-L1 The model registry, versions and aliases

Concept

Experiment tracking (module B14) records every run. The **model registry** names the models you might serve and numbers their versions. In MLflow 3.17:

```
info = mlflow.sklearn.log_model(model, name="model")       # info.model_uri: "models:/m-..."
version = mlflow.register_model(info.model_uri, "survey_topics").version   # 1, then 2, ...
client = MlflowClient()
client.set_registered_model_alias("survey_topics", "champion", "1")
model = mlflow.pyfunc.load_model("models:/survey_topics@champion")
```

An **alias** is a movable name for one version. Serving code loads `@champion` and never names a version. To promote version 2, move the alias. To roll back, move it back. Nothing in the serving code changes.

- One alias points to one version at a time. A version can carry several aliases, such as `champion` and `challenger`.
- `client.get_model_version_by_alias(name, alias).version` tells you which version an alias points to now.
- Loading an alias that does not exist raises `MlflowException: Registered model alias challenger not found.`
- Stages such as `Production` are deprecated since MLflow 2.9. `transition_model_version_stage` warns that stages will be removed. Use aliases.

Resolve the alias once. If you load `@champion` and then ask the registry for the champion's version, someone may move the alias between the two calls. You would serve version 2 and report version 3. Ask for the version first, then load `models:/survey_topics/<version>`, so the model and the reported version always agree.

Keep old versions. A version you delete cannot be a rollback target. In the B27 lab, `train` registers versions 1 and 2, `promote 1` points `champion` at version 1, and `export` writes that version to a folder for the container.

Why it matters

A team served a survey classifier from a file path written into the deployment settings. A retrained model went live by overwriting the file. When accuracy dropped, nobody could say which training run was live or restore the previous file, which had been replaced. With a registry and an alias, the previous version would have been one command away, and every response could have named the version that produced it.

Worked example: weak code

```
(1) model = mlflow.pyfunc.load_model("models:/survey_topics/Production")
(2) version = client.get_latest_versions("survey_topics")[0].version
```

1. **Line 1.** Stages are deprecated since MLflow 2.9. Use an alias.
2. **Line 2.** The newest version is not necessarily the one being served, so this can report the wrong version.

Worked example: strong code

```
 1 import mlflow
 2 from mlflow import MlflowClient
 3 from sklearn.pipeline import Pipeline
 4
 5
 6 def promote(name: str, version: str, alias: str = "champion") -> None:
 7     """Point an alias at a version. Rolling back is promoting the old
   version again.
 8
 9     Raises:
10         ValueError: If alias is empty or version is not a positive whole
   number.
11     """
12     if not alias or not version.isdigit() or int(version) < 1:
13         raise ValueError(f"need an alias and a positive version, got
   {alias!r}, {version!r}")
(1)    MlflowClient().set_registered_model_alias(name, alias, version)
15
16
17 def load_alias(name: str, alias: str = "champion") -> tuple[Pipeline,
   str]:
18     """Load the version an alias points to, and return that version."""
(2)    version = MlflowClient().get_model_version_by_alias(name,
   alias).version
(3)    model = mlflow.sklearn.load_model(f"models:/{name}/{version}")
```

```
21     return model, str(version)
```

1. **Line 14.** Promotion and rollback are the same call with a different version.
2. **Line 19.** Resolve the alias once.
3. **Line 20.** Load that exact version, so the model and the reported version agree.

Register every candidate as a version, serve through an alias, resolve the alias once, and keep old versions as rollback targets.

Recap card

- **Registering a model (MLflow 3.17)** mlflow.register_model(model_uri, name) creates version 1, 2, ... of a named model.
- **Alias** A movable name for one version. set_registered_model_alias(name, alias, version); load models:/name@alias.
- **Promote and roll back** Both move the alias. Keep old versions as rollback targets.
- **Resolve once** get_model_version_by_alias, then load models:/name/version, so the model and the reported version agree.

Practice

Exercise 1. Multiple choice (level 1)

Why does the serving code load `models:/survey_topics@champion` instead of `models:/survey_topics/4`?

- a) Promotion and rollback become moving the alias, with no change to serving code or settings
- b) Loading by alias is faster than loading by version
- c) Aliases keep every version's files, while versions get deleted
- d) MLflow cannot load a model by its version number

Why?

- r1) The alias separates which model is live from the code that serves it.
- r2) Aliases are cached in memory by the tracking server.

Exercise 2. Predict the output (level 2)

This dictionary stands in for a registry's aliases. What does it print?

```
1 aliases = {}
2
3
4 def promote(alias, version):
5     aliases[alias] = version
6
```

```

promote("champion", 1)
promote("challenger", 2)
promote("champion", 2)
promote("champion", 1)
print(aliases, sorted(set(aliases.values())))
```

Exercise 3. Fill the gap (level 2)

Point the champion alias at version 3, then load whatever it points to.

```
client.set_registered_model_alias("survey_topics", "_____(1)", "3")
model = mlflow.pyfunc.load_model("models:/survey_topics_____(2)champion")
```

Exercise 4. Write the function (level 3)

Write `model_uri(name, alias=None, version=None) -> str`.

- With an alias, return `"models:/<name>@<alias>"`. With a version, return `"models:/<name>/<version>"`.
- Raise `ValueError` unless exactly one of `alias` and `version` is given, or if `version` is below 1.

Your code must pass these checks:

- An alias follows @.
- A version follows a slash.
- Raise ValueError unless exactly one of alias and version is given.
- Versions start at 1. Test for None with `is None`, so version 0 reaches the check.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# A registry URI for a model by alias or by version.
def model_uri(
    name: str,
    alias: str | None = None,
    version: int | None = None,
) -> str: ...
```

Starter code:

```
def model_uri(name, alias=None, version=None):
    return f"models:/{name}/{alias}"
```

Exercise 5. Write the function (level 3)

Every alias move is logged as `(alias, version)` in time order. Write `rollback_target(moves, alias="champion") -> int | None`: the version `alias` pointed to before its current one.

- Ignore moves of other aliases.
- Skip moves that left the alias on the version it already had.
- Return `None` if the alias has never pointed to a different version.

Your code must pass these checks:

- Follow only the alias you were asked about: champion went 1 then 2.
- A repeat move to the same version is not a change. Skip it.
- Return None when the alias never pointed to a different version.
- Use the alias argument.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# The version an alias pointed to before its current one.
def rollback_target(
    moves: list[tuple[str, int]],
    alias: str = 'champion',
) -> int | None: ...
```

Starter code:

```python
def rollback_target(moves, alias="champion"):
    return moves[-2][1]
```

Exercise 6. Spot the bug (level 4)

Responses sometimes report version 3 while version 2's predictions come back. The champion alias was being moved during a deployment. Tap the line responsible.

```python
import mlflow
from mlflow import MlflowClient


def load_champion(name):
    model = mlflow.sklearn.load_model(f"models:/{name}@champion")
    version = MlflowClient().get_model_version_by_alias(name,
    "champion").version
    return model, version
```

Write down the line number of the bug.

Sources

- MLflow 3.17.2: log_model(model, name='model', registered_model_name=...) returns ModelInfo with model_uri 'models:/m-...'; register_model(model_uri, name) returns ModelVersion with an integer version; set_registered_model_alias(name, alias, version); models:/name@alias loads; a moved alias returns the new version; a missing alias raises MlflowException 'Registered model alias challenger not found.'; transition_model_version_stage warns it is deprecated since 2.9.0 and stages will be removed. (verified: Run locally on MLflow 3.17.2 with a SQLite tracking store, November 2026.)

B27-L2 Serving a model with FastAPI

Concept

A prediction service is a small web API. FastAPI builds one from Python functions and pydantic models. In FastAPI 0.162 with pydantic 2.14:

```
class PredictRequest(BaseModel):
    texts: list[Annotated[str, Field(min_length=1, max_length=2_000)]] = Field(min_length=1)

@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest) -> PredictResponse: ...
```

The schema is the contract. FastAPI checks every request against `PredictRequest` before your function runs. A missing field, an empty list or an empty string gets status 422 with a `detail` list naming the problem, such as `too_short` at `("body", "texts")`. Your function only sees valid input. `response_model` checks what goes back.

Load the model once, at start-up. Loading inside the endpoint repeats the work on every request. Loading at import time runs on import, including in tests. Use a lifespan function, which runs when the server starts:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model, app.state.meta = loader()
    yield
```

Pass `loader` into an app factory, `create_app(loader)`, so tests hand in a fake model and production hands in the real one.

Health and status codes. `GET /health` answers 200 once the model has loaded and 503 before, so the platform sends traffic only to a ready copy. Return 413 when a request holds more texts than you allow.

Testing. `TestClient(app)` from `fastapi.testclient` calls the app without a server. It needs httpx, and Starlette 1.10 prefers the `httpx2` package and warns with `httpx`. Use it as `with TestClient(app) as client:`. Without `with`, the lifespan never runs: in a check, `/health` returned 503.

Why it matters

A team's first prediction service loaded a 300 MB model inside the endpoint. Each request took over a second, and memory climbed under load until the service restarted. Moving the load into a lifespan function brought requests down to milliseconds. A test with `TestClient` used as a context manager now proves that the model loads once.

Worked example: weak code

```
1  app = FastAPI()
1  model = joblib.load("model/model.joblib")
3
4  @app.post("/predict")
2  def predict(payload: dict):
6      return {"labels": model.predict(payload["texts"]).tolist()}
```

1. **Line 2.** Loads at import time: importing the module in a test needs the real model file.
2. **Line 5.** A plain dict skips validation. A missing key becomes a 500 error instead of a clear 422.

Worked example: strong code

```
1  from collections.abc import AsyncIterator
2  from contextlib import asynccontextmanager
3
4  from fastapi import FastAPI, HTTPException
5
6
7  def create_app(loader: Loader, max_batch: int = 100) -> FastAPI:
8      """Build the API. The model loads once, at start-up."""
9
10     @asynccontextmanager
11     async def lifespan(app: FastAPI) -> AsyncIterator[None]:
1          app.state.model, app.state.meta = loader()
13         yield
14
15     app = FastAPI(title="Survey topics", lifespan=lifespan)
16
17     @app.get("/health")
18     def health() -> dict[str, str]:
19         if getattr(app.state, "model", None) is None:
```

```
(2)            raise HTTPException(status_code=503, detail="model not
    loaded")
21         return {"status": "ok", "model_version": app.state.meta.version}
22
23     @app.post("/predict", response_model=PredictResponse)
(3)    def predict(request: PredictRequest) -> PredictResponse:
25         if len(request.texts) > max_batch:
(4)            raise HTTPException(status_code=413, detail=f"at most
    {max_batch} texts per request")
27         probs = app.state.model.predict_proba(request.texts)
28         predictions = [
29             Prediction(label=str(app.state.model.classes_[row.argmax()]),
    confidence=round(float(row.max()), 4))
30             for row in probs
31         ]
32         return PredictResponse(predictions=predictions,
    model_name=app.state.meta.name, model_version=app.state.meta.version)
33
34     return app
```

1. **Line 12.** Runs once when the server starts. Tests pass a fake loader.
2. **Line 20.** Not ready yet: the platform keeps traffic away.
3. **Line 24.** FastAPI validates against PredictRequest first and answers 422 for bad input.
4. **Line 26.** A limit you choose, checked in code: 413 means the request is too large.

Let pydantic schemas validate requests, load the model once in a lifespan function inside an app factory, and test with TestClient as a context manager.

Recap card

- **Request schema (FastAPI 0.162, pydantic 2.14)** A pydantic model with Field rules. Invalid input gets 422 with a detail list before your function runs.
- **Loading the model** Once, in a lifespan function that stores it on app.state. Not per request, not at import time.
- **Status codes** 503 from /health until the model loads, 422 for invalid input, 413 for too many texts.
- **TestClient** with TestClient(app) as client: runs the lifespan. Without with, start-up never runs.

Practice

Exercise 1. Multiple choice (level 1)

Why does the service load its model in a lifespan function instead of inside the `/predict` function?

- a) It loads once when the server starts, instead of on every request
- b) FastAPI forbids file access inside endpoints
- c) The lifespan function validates the model's schema
- d) It makes the model load at import time

Why?

- r1) Loading is slow and uses memory; doing it per request multiplies both by the number of requests.
- r2) Lifespan functions run on a faster thread.

Exercise 2. Predict the output (level 2)

What does this print?

What you need to know.

`Annotated[str, Field(min_length=1)]` attaches a rule to a type, here to each item of a list. `Model.model_validate(data)` checks a dictionary, and a failure raises `ValidationError`, whose `errors()` lists each problem with a `type` and a `loc`:

```python
class Tag(BaseModel):
    name: Annotated[str, Field(min_length=2)]

Tag.model_validate({"name": "x"})     # ValidationError: string_too_short at ('name',)
```

```python
from typing import Annotated

from pydantic import BaseModel, Field, ValidationError


class PredictRequest(BaseModel):
    texts: list[Annotated[str, Field(min_length=1)]] = Field(min_length=1, max_length=3)


for payload in [{"texts": ["late"]}, {"texts": []}, {"texts": ["a", ""]}]:
    try:
        PredictRequest.model_validate(payload)
        print("ok")
    except ValidationError as e:
        err = e.errors()[0]
```

```
        print(err["type"], err["loc"])
```

Exercise 3. Fill the gap (level 2)

Declare the prediction route, and report that the model has not loaded yet.

```
@app._____(1)("/predict", response_model=PredictResponse)
def predict(request: PredictRequest) -> PredictResponse: ...

@app.get("/health")
def health() -> dict[str, str]:
    if getattr(app.state, "model", None) is None:
        raise HTTPException(status_code=_____(2), detail="model not loaded")
    return {"status": "ok"}
```

Exercise 4. Write the function (level 3)

Write `request_status(texts, max_batch) -> int`, the status the API gives a request body's `texts` value before predicting.

- 422 unless `texts` is a non-empty list whose items are all non-empty strings.
- 413 if it holds more than `max_batch` items.
- 200 otherwise.

What you need to know.

`isinstance(value, kind)` checks a value's type, and `all` is true when every item passes:

```
isinstance("a", str), isinstance(["a"], list)       # (True, True)
all(isinstance(x, int) for x in [1, 2, "3"])         # False
```

Your code must pass these checks:

- A list of 1 to max_batch non-empty strings is valid.
- Return 422 for an empty list, an empty string, a non-string item, or a value that is not a list.
- Return 413 above max_batch, and 200 at exactly max_batch.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# HTTP status for a request's texts value, before any prediction.
def request_status(texts: object, max_batch: int) -> int: ...
```

Starter code:

```
def request_status(texts, max_batch):
    return 200
```

Exercise 5. Write the function (level 3)

Write `to_predictions(probs, classes) -> list[dict]`, the response body for a batch.

- `probs` has one row per text and one column per class, in the order of `classes`.
- For each row return `{"label": <class with the highest probability>, "confidence": <that probability rounded to 4 places>}`.
- Raise `ValueError` if the number of columns differs from `len(classes)`.

Your code must pass these checks:

- Take the class with the highest probability in each row.
- Round the confidence to 4 places and return a plain float.
- Raise ValueError when the columns do not match the classes.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Label and confidence for each row of class probabilities.
def to_predictions(
    probs: np.ndarray,
    classes: list[str],
) -> list[dict[str, str | float]]: ...
```

Starter code:

```python
import numpy as np


def to_predictions(probs, classes):
    return [{"label": classes[0], "confidence": 1.0} for _ in probs]
```

Exercise 6. Spot the bug (level 4)

Each request to this service takes about 900 ms, almost all spent before the model predicts. Tap the line responsible.

```python
import joblib
from fastapi import FastAPI

app = FastAPI()


@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest) -> PredictResponse:
    model = joblib.load("model/model.joblib")
    probs = model.predict_proba(request.texts)
    return build_response(probs, model.classes_)
```

Write down the line number of the bug.

Sources

- FastAPI 0.162.0 with pydantic 2.14.2 and Starlette 1.10.0: an empty list for a Field(min_length=1) list returns 422 with type too_short; a missing field returns missing at ('body', 'texts'); a string for a list returns list_type; GET on a POST route returns 405; /openapi.json and /docs are served; TestClient without a with block does not run the lifespan (/health returned 503), with a with block it does; starlette.testclient warns 'Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.' (verified: Run locally on FastAPI 0.162.0, pydantic 2.14.2, Starlette 1.10.0 and httpx 0.28.1, November 2026.)
- The httpx2 package that Starlette 1.10 recommends for TestClient. (unverified: Named in Starlette 1.10.0's source and FastAPI 0.162.0's extras; not installed or tested in the build session.)

B27-L3 Packaging a model service as a container image

Concept

A container image holds your code, its dependencies and the model, so the service runs the same on your laptop and in the cloud. Build it from a `Dockerfile`:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements-serve.txt .
RUN pip install --no-cache-dir -r requirements-serve.txt
COPY src/ src/
COPY model/ model/
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
CMD ["uvicorn", "serve_lab.app:build", "--factory", "--host", "0.0.0.0",
    "--port", "8000"]
```

- **Pin exact versions** in the requirements file, such as `scikit-learn==1.9.1`. A model saved with one scikit-learn version may not load, or may predict differently, with another. Install only what serving needs: the lab's image left MLflow out.
- **Order for the cache.** Docker reuses each step until its inputs change. Copy the requirements and install them before copying code, so a code change does not reinstall every package.

- **Bind to 0.0.0.0.** Uvicorn 0.54 binds to 127.0.0.1 by default, which accepts connections only from inside the container. `EXPOSE` documents the port but does not change the bind address.
- **Run as a non-root user**, so a flaw in the service cannot change the image's files.
- **Settings from environment variables**, checked when the app starts: `MODEL_DIR`, `MAX_BATCH`. Secrets never go in the image. The platform injects them at run time (lesson B27-L4).
- **Load only model files you built.** `joblib.load` and `pickle` run code stored in the file.

`uvicorn serve_lab.app:build --factory` calls `build()` when the server starts, so importing the module loads nothing. Add a `.dockerignore` so virtual environments and test data stay out.

In the B27 lab, Docker 29.9 built this image at 561 MB. It answered `/predict` as `appuser` and logged one JSON line per request.

Why it matters

A team's model service worked on the developer's laptop and returned different predictions in production. The requirements file listed `scikit-learn` with no version, so the production build installed a newer release than the one that saved the model. Pinning the version that trained the model, and installing from the same file in both places, made the predictions match.

Worked example: weak code

```
1 FROM python:3.12
1 COPY . .
2 RUN pip install fastapi uvicorn scikit-learn
3 CMD ["uvicorn", "serve_lab.app:build", "--factory"]
```

1. **Line 2.** Copies everything before installing, so every code change reinstalls all packages. It also copies .venv and test data.
2. **Line 3.** No versions: next month's build installs different releases.
3. **Line 4.** Default host 127.0.0.1: unreachable from outside the container. Also runs as root.

Worked example: strong code

```
1 FROM python:3.12-slim
1 ENV PYTHONUNBUFFERED=1 MODEL_DIR=/app/model
3 WORKDIR /app
4 COPY requirements-serve.txt .
2 RUN pip install --no-cache-dir -r requirements-serve.txt
```

```
 6 COPY src/ src/
 7 COPY model/ model/
 8 ENV PYTHONPATH=/app/src
 9 RUN useradd --create-home appuser
(3) USER appuser
11 EXPOSE 8000
(4) CMD ["uvicorn", "serve_lab.app:build", "--factory", "--host", "0.0.0.0",
      "--port", "8000"]
```

1. **Line 2.** Logs appear at once; the app reads MODEL_DIR at start-up.
2. **Line 5.** Pinned versions, installed before the code, so this layer is reused until the requirements change.
3. **Line 10.** The service runs without root rights.
4. **Line 12.** Listens on every interface, so the published port reaches it.

Pin serving dependencies, install them before copying code, bind to 0.0.0.0, run as a non-root user, and read settings from the environment at start-up.

Recap card

- **Pinned serving requirements** name==version for each package the service needs, and nothing else. The model's scikit-learn version must match.
- **Dockerfile order** FROM, copy requirements, install, copy code and model, non-root USER, CMD. Docker reuses steps until an input changes.
- **--host 0.0.0.0** Uvicorn 0.54 defaults to 127.0.0.1, which only the container itself can reach.
- **Settings and secrets** Read settings from environment variables and check them at start-up. Secrets are injected at run time, never built in.

Practice

Exercise 1. Multiple choice (level 1)

The container logs `Uvicorn running on http://127.0.0.1:8000`, but `curl localhost:8000/health` from your laptop gets no answer, even with `-p 8000:8000`. Why?

- a) 127.0.0.1 inside the container accepts connections only from inside the container. Start uvicorn with --host 0.0.0.0
- b) The Dockerfile has no EXPOSE line
- c) Port 8000 is reserved by Docker
- d) The model has not finished loading

Why?

- r1) The published port forwards to the container's network interface, where nothing is listening.
- r2) EXPOSE opens the port in the container's firewall.

Exercise 2. Predict the output (level 2)

Settings arrive as strings from the environment. What does this print?

```python
def parse_max_batch(raw):
    if not raw.isdigit() or not 1 <= int(raw) <= 1000:
        raise ValueError(f"MAX_BATCH must be 1 to 1000, got {raw!r}")
    return int(raw)


for raw in ["100", "0", "50.5", " 20"]:
    try:
        print(parse_max_batch(raw))
    except ValueError:
        print("rejected", repr(raw))
```

Exercise 3. Put in order (level 2)

Order the Dockerfile steps so a code change rebuilds as little as possible and the service runs without root.

- CMD ["uvicorn", ..., "--host", "0.0.0.0"]
- RUN useradd --create-home appuser, then USER appuser
- COPY src/ src/ and COPY model/ model/
- COPY requirements-serve.txt .
- RUN pip install --no-cache-dir -r requirements-serve.txt
- FROM python:3.12-slim

Exercise 4. Write the function (level 3)

Write `parse_requirements(lines) -> dict[str, str]` for a serving requirements file.

- Skip blank lines and lines starting with `#`, after stripping spaces.
- Every other line must be `name==version`. Return `{name: version}`.
- Raise `ValueError` naming the first line that is not pinned with `==`.

Your code must pass these checks:

- Skip comments and blank lines, strip spaces, and split each pin on ==.
- Raise ValueError, naming the line, for anything not pinned as name==version.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Exact pins from a requirements file.
def parse_requirements(lines: list[str]) -> dict[str, str]: ...
```

Starter code:

```python
def parse_requirements(lines):
    return {}
```

Exercise 5. Write the function (level 3)

Write `load_settings(env) -> dict` from a dictionary of environment variables.

- `model_dir`: `env["MODEL_DIR"]`, default `"model"`.
- `max_batch`: `env["MAX_BATCH"]` as an int, default 100. It must be all digits and from 1 to 1,000, else raise `ValueError` naming `MAX_BATCH`.
- Raise `ValueError` if `model_dir` is empty.

Your code must pass these checks:

- Use the defaults when a variable is not set.
- Read both variables and convert MAX_BATCH to an int.
- Reject MAX_BATCH unless it is a whole number from 1 to 1000, and name the variable.
- Raise ValueError for an empty MODEL_DIR.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Service settings from environment variables, checked at start-up.
def load_settings(
    env: dict[str, str],
) -> dict[str, str | int]: ...
```

Starter code:

```python
def load_settings(env):
    return {"model_dir": env["MODEL_DIR"], "max_batch":
    int(env["MAX_BATCH"])}
```

Exercise 6. Spot the bug (level 4)

This image works, but every one-line code change makes `docker build` reinstall all packages, which takes four minutes. Tap the line responsible.

```dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements-serve.txt
RUN useradd --create-home appuser
USER appuser
```

```
7 CMD ["uvicorn", "serve_lab.app:build", "--factory", "--host", "0.0.0.0",
    "--port", "8000"]
```

Write down the line number of the bug.

Sources

- Docker 29.9.2 built the lab's Dockerfile on python:3.12-slim into a 561 MB image that ran as appuser, answered /health and /predict, returned 422 for an empty list, and logged one JSON line per request; uvicorn 0.54.0 --help gives --host default 127.0.0.1 and a --factory flag. (verified: Built and run locally, November 2026. The base image came from the mirror.gcr.io copy because Docker Hub returned 429 Too Many Requests.)
- A server bound to 127.0.0.1 inside a container is unreachable through a published port; EXPOSE documents a port and does not change the bind address. (search-only: Search results (uvicorn GitHub discussion 2325 and Stack Overflow answers), November 2026. Bridge networking was not available in the build session to test it.)

B27-L4 Deploying the container to Azure Container Apps

Concept

Azure Container Apps runs a container image behind an HTTPS address and scales it for you. One Azure CLI command deploys from your lab folder (Microsoft Learn, November 2025):

```
1 az login
2 az extension add --name containerapp --upgrade
3 az containerapp up --name survey-topics --source . --ingress external --
    target-port 8000
```

With `--source`, `up` creates a resource group, a Container Apps environment with a Log Analytics workspace, and an Azure Container Registry. It builds the image from your Dockerfile, pushes it, deploys it and prints the app's URL. Redeploy with the same command plus `--resource-group` and `--environment`.

Secrets. Define them on the app with `--secrets "api-key=<value>"`, or better as Key Vault references. Pass them to the container as environment variables with `--env-vars "MAX_BATCH=50" "API_KEY=secretref:api-key"`. Changing a secret does not create a new revision: restart the revision or deploy a new one to pick it up.

Health probes. Container Apps has startup, liveness and readiness probes. An HTTP probe passes on a status from 200 to 399. With ingress on, the default probes are TCP checks on the target port, which pass as soon as uvicorn listens, before the model has loaded. Point an HTTP readiness probe at `/health`, which answers 503 until the model is ready.

Scaling. With no rule of your own, an app scales on HTTP traffic from 0 to 10 replicas, adding a replica per 10 concurrent requests: `ceil(concurrent / 10)`. At zero replicas you pay no usage charges, but the next request waits for a container to start and load the model. Set `--min-replicas 1` when that wait matters.

Revisions. A change to the container template, such as a new image or scale rule, creates a new, immutable revision. Lesson B27-L6 uses revisions to roll back.

Delete the resource group when you finish practising, or it keeps costing money.

Why it matters

A team deployed a model service with the default settings and saw the first request each morning time out. The app had scaled to zero overnight, and the cold start, including a 40-second model load, outlasted the caller's timeout. Setting a minimum of one replica for working hours removed the timeouts, and an HTTP readiness probe on `/health` stopped traffic reaching replicas whose model was still loading.

Worked example: weak code

```
az containerapp up --name survey-topics --source . --ingress external --
  target-port 8000 --env-vars "API_KEY=sk-live-4f9a" "MAX_BATCH=100"
# readiness: default TCP probe on port 8000
```

1. **Line 1.** The key sits in plain text in the command, the shell history and the app's settings.
2. **Line 2.** TCP passes once the port opens, before the model loads, so early requests can fail.

Worked example: strong code

```
az containerapp up --name survey-topics --source . --ingress external --
  target-port 8000 --resource-group rg-survey --environment env-survey
az containerapp secret set --name survey-topics --resource-group rg-survey
  --secrets "api-
  key=keyvaultref:<KEY_VAULT_SECRET_URI>,identityref:<IDENTITY_ID>"
```

```
❷ az containerapp update --name survey-topics --resource-group rg-survey --
    set-env-vars "MAX_BATCH=100" "API_KEY=secretref:api-key" --min-replicas
    1 --max-replicas 5
❸ # readiness: HTTP GET /health on port 8000; 503 until the model loads
```

1. **Line 2.** The value lives in Key Vault. The app reads it through a managed identity.
2. **Line 3.** secretref: points the variable at the secret; one warm replica avoids cold starts.
3. **Line 4.** Traffic reaches a replica only when its model is ready.

Deploy with az containerapp up, keep secrets in Key Vault references passed as secretref variables, probe /health over HTTP, and choose min replicas for your latency needs.

Recap card

- **az containerapp up --source .** Creates the resource group, environment, Log Analytics and a registry, builds from the Dockerfile, deploys and prints the URL.
- **Secrets in Container Apps** --secrets name=value or a Key Vault reference; --env-vars NAME=secretref:name. A secret change needs a restart or a new revision.
- **Probes** Startup, liveness, readiness. HTTP passes on 200 to 399. The default with ingress is TCP, which passes before the model loads.
- **Default scaling** HTTP rule, 0 to 10 replicas, 10 concurrent requests each. Scale to zero costs nothing but adds a cold start.

Practice

Exercise 1. Multiple choice (level 1)

With ingress on and no probes of your own, why can the first requests to a new replica fail?

- a) The default readiness probe is a TCP check, which passes when the port opens, before the model has loaded
- b) Container Apps sends no traffic until a liveness probe fails
- c) TCP probes count a 503 response as success
- d) New replicas never receive traffic in single revision mode

Why?

- r1) A TCP probe only checks that something listens. An HTTP probe on /health checks that the model is ready.

- r2) TCP probes run only once a day.

Exercise 2. Predict the output (level 2)

Container Apps sizes an HTTP-scaled app as `ceil(concurrent requests / target)`, kept between the minimum and maximum replicas. What does this print?

```python
import math


def replicas(concurrent, target=10, min_r=0, max_r=10):
    return max(min_r, min(max_r, math.ceil(concurrent / target)))


print([replicas(c) for c in (0, 7, 35, 250)], replicas(0, min_r=1))
```

Exercise 3. Fill the gap (level 2)

Deploy from the current folder so the app is reachable from the internet on the port uvicorn listens on.

```
az containerapp up --name survey-topics --source . --ingress _____(1) --
    target-port _____(2)
```

Exercise 4. Write the function (level 3)

Write `env_var_args(plain, secret_refs) -> list[str]`, the items for `--env-vars`.

- `plain` maps variable names to values: `"MAX_BATCH=50"`.
- `secret_refs` maps variable names to secret names: `"API_KEY=secretref:api-key"`.
- Return all items sorted by variable name.
- Raise `ValueError` if a variable appears in both, or any variable name is empty.

Your code must pass these checks:

- Secrets become NAME=secretref:secret-name; sort every item by variable name.
- Raise ValueError when a variable is both plain and a secret.
- Raise ValueError for an empty variable name.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Items for az containerapp --env-vars, secrets as secretref: references.
def env_var_args(
    plain: dict[str, str],
```

```
    secret_refs: dict[str, str],
) -> list[str]: ...
```

Starter code:

```
def env_var_args(plain, secret_refs):
    return [f"{k}={v}" for k, v in {**plain, **secret_refs}.items()]
```

Exercise 5. Write the function (level 3)

Write `revision_name(app, suffix) -> str`, returning `"<app>-<suffix>"` after checking the suffix against the Container Apps rules.

- Only lower-case letters, digits and dashes.
- Starts with a letter and ends with a letter or digit.
- No two dashes in a row, and at most 64 characters.
- Raise `ValueError` naming the suffix if any rule fails.

What you need to know.

`str.isalnum()` is true when every character is a letter or a digit, and `str.islower()` when no character is upper case:

```
"v2".isalnum(), "v2".islower(), "V2".islower()    # (True, True, False)
```

Your code must pass these checks:

- Join the app name and a valid suffix with a dash.
- Reject upper case, a leading digit, a trailing dash, two dashes in a row, other characters, empty and over 64 characters.
- 64 characters is allowed.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# A revision name, after checking the suffix rules.
def revision_name(app: str, suffix: str) -> str: ...
```

Starter code:

```
def revision_name(app, suffix):
    return f"{app}-{suffix}"
```

Exercise 6. Architecture design (level 4)

Plan the Container Apps deployment of the survey classifier. Callers time out after 10 seconds, and loading the model takes 15.

Options: --min-replicas 1, so one replica is always warm, The default of 0, to pay nothing when idle, --min-replicas 10, the maximum, at all times.

1. Minimum replicas
2. The API key the service checks

3. Readiness probe

Sources

- Azure Container Apps: az containerapp up with --source builds from the Dockerfile if present, creating a resource group, environment, Log Analytics workspace and Container Registry, and prints the URL; EXPOSE sets the target port; redeploy needs --resource-group and --environment. (search-only: Microsoft Learn, 'Deploy Azure Container Apps with the az containerapp up command', ms.date 2025-11-20, read November 2026. Not run: no Azure subscription in the build session.)
- Container Apps secrets: --secrets name=value, Key Vault references as keyvaultref:<uri>,identityref:<id>, --env-vars NAME=secretref:name; adding or changing a secret creates no revision, so restart or deploy a new revision. (search-only: Microsoft Learn, 'Manage secrets in Azure Container Apps', ms.date 2026-09-04, read November 2026.)
- Container Apps probes: startup, liveness and readiness; HTTP success is a status from 200 to 399; with ingress the default probes are TCP on the target port. Scaling: default HTTP rule with min 0 and max 10 replicas, concurrentRequests default 10, desiredReplicas = ceil(metric / target), no usage charges at zero replicas. (search-only: Microsoft Learn, 'Health probes in Azure Container Apps' (ms.date 2025-11-06) and 'Scaling in Azure Container Apps' (ms.date 2026-05-19), read November 2026.)
- Revision suffix rules: lower-case alphanumerics or dashes, starting with a letter, ending with an alphanumeric, no two consecutive dashes, at most 64 characters; revision-scope changes (image, scale rules) create a revision, application-scope changes (secrets, ingress, traffic) do not. (search-only: Microsoft Learn, 'Update and deploy changes in Azure Container Apps', ms.date 2025-10-27, read November 2026.)
- Azure CLI: az containerapp secret set --secrets accepts key=value and key=keyvaultref:<url>,identityref:<id> (GA); az containerapp update takes --set-env-vars (a value starting secretref: reads a secret), --min-replicas and --max-replicas; az containerapp up takes --name, --source, --ingress, --target-port, --env-vars, --resource-group and --environment; a container app name must be under 32 characters. (search-only: Azure CLI reference on Microsoft Learn, 'az containerapp' and 'az containerapp secret' (ms.date 2026-10-23), read November 2026.)

B27-L5 Monitoring a served model for drift

Concept

A deployed model gets worse quietly. True labels often arrive weeks later, if at all, so watch what you can see now:

- **Inputs**: the distribution of each feature, such as answer length.
- **Outputs**: the share of each predicted class, and the model's confidence.
- **Service**: request count, error rate and latency. Report the 95th percentile, not the mean, because a few slow requests hide in an average.

Log one structured record per request: model version, number of texts, mean confidence and latency. Leave the texts out: open answers can hold personal data.

Population stability index (PSI) compares a feature's distribution now with a reference, such as the training data. Cut the reference into bins at its quantiles, so each holds about the same share. Count the share of reference values `e` and current values `a` in each bin:

```
psi = sum((a - e) * ln(a / e))      # over bins
```

Identical shares give 0. Raise any share below a small floor, such as 0.0001, before the logarithm, or an empty bin gives infinity. With two bins, reference shares 0.5 and 0.5 against current shares 0.25 and 0.75 give 0.2747.

A common rule of thumb reads below 0.1 as stable, 0.1 to 0.25 as a moderate shift, and above 0.25 as a large one. It dates from credit scoring and has no statistical basis. PSI also depends on the number of bins and the sample sizes. Keep the bin edges fixed, and tune the thresholds against your own past incidents.

In the B27 lab, a later batch with longer answers and more delivery complaints gave a PSI of 4.708 on answer length and 0.687 on confidence. A fresh batch from the training distribution gave 0.008.

Drift is a reason to look, not proof the model is wrong. Confirm with labelled samples before you retrain (lesson B27-L6).

Why it matters

A retailer's complaint classifier kept its accuracy on the test set while customer service reported more misrouted complaints. A new mobile app had changed how customers wrote: shorter answers and words the model had never seen. A weekly PSI check on answer length and confidence would

have flagged the change in its first week, two months before the complaints did.

Worked example: weak code

```
LOGGER.info(f"predicted {labels} for {request.texts}")
shift = abs(np.mean(current) - np.mean(reference)) > 0.1
```

1. **Line 1.** Free text that is hard to analyse, and it stores respondents' answers in the logs.
2. **Line 2.** A mean misses changes in spread and shape, and 0.1 means nothing without units.

Worked example: strong code

```
import numpy as np

EPS = 1e-4


def psi(expected: np.ndarray, actual: np.ndarray, bins: int = 10) -> float:
    """Population stability index of actual against expected, on expected's quantile bins.

    Raises:
        ValueError: If bins is below 2 or either array is empty.
    """
    if bins < 2:
        raise ValueError(f"bins must be at least 2, got {bins}")
    if len(expected) == 0 or len(actual) == 0:
        raise ValueError("expected and actual must not be empty")
    inner = np.unique(np.quantile(expected, np.linspace(0, 1, bins + 1))[1:-1])
    n_bins = len(inner) + 1
    e = np.bincount(np.searchsorted(inner, expected, side="right"), minlength=n_bins) / len(expected)
    a = np.bincount(np.searchsorted(inner, actual, side="right"), minlength=n_bins) / len(actual)
    e, a = np.clip(e, EPS, None), np.clip(a, EPS, None)
    return float(np.sum((a - e) * np.log(a / e)))
```

1. **Line 16.** Inner edges from the reference only. unique drops repeated edges from tied values.
2. **Line 18.** searchsorted gives each value's bin; the outer bins are open, so no value falls outside.
3. **Line 20.** An empty bin would make ln(a / e) infinite.

Log structured records without raw text, compare inputs and outputs with a fixed-bin PSI, treat 0.1 and 0.25 as starting points, and confirm drift with labels.

Recap card

- **What to monitor without labels** Input distributions, predicted class shares, confidence, error rate and p95 latency.
- **PSI** sum((a - e) * ln(a / e)) over the reference's quantile bins, with shares floored at a small value.
- **PSI bands** Below 0.1 stable, 0.1 to 0.25 moderate, above 0.25 large: a rule of thumb from credit scoring. Tune it to your data.
- **Prediction logs** One structured record per request with version, counts, confidence and latency. Never the respondent's text.

Practice

Exercise 1. Multiple choice (level 1)

Labels for your served model arrive three months late. What can you monitor this week?

- a) Input feature distributions, predicted class shares, confidence, error rate and latency
- b) Nothing: without labels, monitoring is impossible
- c) Accuracy on the original test set
- d) The training loss

Why?

- r1) These need no labels and change as soon as the incoming data changes.
- r2) They measure accuracy directly.

Exercise 2. Predict the output (level 2)

What does this print?

```python
import math

e = [0.5, 0.5]
a = [0.25, 0.75]
psi = sum((ai - ei) * math.log(ai / ei) for ai, ei in zip(a, e))
print(round(psi, 4))
```

Exercise 3. Fill the gap (level 2)

Complete the last two lines of the PSI calculation.

```
e, a = np.clip(e, EPS, None), np.clip(a, _____(1), None)
return float(np.sum((a - e) * np._____(2)(a / e)))
```

Exercise 4. Write the function (level 3)

Write `psi(expected, actual, bins=10) -> float`.

- Inner bin edges: `np.quantile(expected, np.linspace(0, 1, bins + 1))[1:-1]`, with repeats removed by `np.unique`.
- Shares per bin for each array, using `np.searchsorted(inner, values, side="right")` and `np.bincount(..., minlength=len(inner) + 1)`.
- Raise every share below `1e-4` to `1e-4`, then return `sum((a - e) * ln(a / e))` as a float.
- Raise `ValueError` if `bins` is below 2 or either array is empty.

What you need to know.

`np.searchsorted(edges, values, side="right")` gives, for each value, how many edges are at or below it, which is its bin number. `np.bincount` then counts each bin number:

```
edges = np.array([10.0, 20.0])
idx = np.searchsorted(edges, np.array([5.0, 10.0, 15.0, 25.0]),
    side="right")   # [0, 1, 1, 2]
np.bincount(idx,
    minlength=3)
    # [1, 2, 1]
```

Your code must pass these checks:

- Identical distributions give 0.
- Use the quantile edges of expected and the formula sum((a - e) * ln(a / e)).
- Floor the shares at 1e-4, so empty bins do not give infinity.
- Raise ValueError for bins below 2 or an empty array.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
EPS = ...

# Population stability index of actual against expected.
def psi(
    expected: np.ndarray,
    actual: np.ndarray,
    bins: int = 10,
```

```
) -> float: ...
```

Starter code:

```
import numpy as np


def psi(expected, actual, bins=10):
    return 0.0
```

Exercise 5. Write the function (level 3)

Each request logs a record such as `{"n_texts": 4, "mean_confidence": 0.91, "latency_ms": 12.5}`. Write `summarise(records) -> dict` for a monitoring report.

- `requests`: the number of records. `texts`: the total of `n_texts`.
- `p95_latency_ms`: `float(np.percentile(latencies, 95))`, rounded to 2 places.
- `mean_confidence`: the mean confidence per text, weighting each record by its `n_texts`, rounded to 4 places.
- Raise `ValueError` if there are no records or no texts.

Your code must pass these checks:

- Weight each record's confidence by its n_texts: (0.5 * 1 + 0.9 * 3) / 4 = 0.8.
- Report the 95th percentile latency with np.percentile, rounded to 2 places.
- Raise ValueError when there are no records or no texts.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Request count, text count, p95 latency and text-weighted mean
    confidence.
def summarise(
    records: list[dict[str, float]],
) -> dict[str, float]: ...
```

Starter code:

```
import numpy as np


def summarise(records):
    return {"requests": len(records)}
```

Exercise 6. Spot the bug (level 4)

This drift check reports a PSI near 0 even when the current data has clearly moved. Tap the lines responsible, then fix them.

```python
import numpy as np


def psi(expected, actual, bins=10):
    q = np.linspace(0, 1, bins + 1)
    e_inner = np.quantile(expected, q)[1:-1]
    a_inner = np.quantile(actual, q)[1:-1]
    e = np.bincount(np.searchsorted(e_inner, expected, side="right"), minlength=bins) / len(expected)
    a = np.bincount(np.searchsorted(a_inner, actual, side="right"), minlength=bins) / len(actual)
    e, a = np.clip(e, 1e-4, None), np.clip(a, 1e-4, None)
    return float(np.sum((a - e) * np.log(a / e)))
```

Write down the line number of the bug. Then write the fix.

Sources

- The PSI bands 0.1 and 0.25 are an industry rule of thumb (Lewis, 1994) without statistical derivation; PSI's distribution depends on the number of bins and the sample sizes. (search-only: Search results including the Journal of Risk Model Validation paper on PSI's statistical properties and arXiv 2303.01227, November 2026.)
- B27 lab: PSI of answer length 4.708 and of confidence 0.687 on a shifted batch of 1,000 answers, and 0.008 for confidence on an unshifted batch, using 10 quantile bins and a 1e-4 floor. (verified: labs/serving reference run, NumPy 2.5.4 and scikit-learn 1.9.1, November 2026.)

B27-L6 Retraining, promotion and rollback

Concept

A served model needs a plan for its replacement. Decide in advance what starts a retrain: a schedule, a drift alert confirmed by a labelled sample (lesson B27-L5), or a fall in accuracy once labels arrive.

Compare like with like. Score the new model, the **challenger**, and the live one, the **champion**, on the same recent labelled data. Each model's own validation score was measured on different data and proves nothing about which is better. Use several batches, and promote only when the challenger wins by a margin worth the risk of a change.

Shadow first. Keep serving the champion's answers, run the challenger on the same requests, and log both. The agreement rate, and which classes the two models swap, show how much the change would affect users before any user sees it.

Promote in steps you can reverse.

- Move the registry alias to the challenger's version (lesson B27-L1), export it, and build an image tagged with that version, such as `survey-topics:3`.
- Deploy a new Container Apps revision. In the default single revision mode, traffic moves once the new revision passes its readiness probe, and the old revision is shut down. In multiple revision mode, the old revision stays active, and you can split traffic, for example 80% to the old and 20% to the new.

Roll back by reversing the same steps: move the alias back, and send traffic back to the previous revision or redeploy the previous image tag. Container Apps keeps up to 100 inactive revisions. Keep old registry versions and old image tags for the same reason.

Write every promotion down: which versions, who, when, and the evidence. In the B27 lab, `promote 1` after version 2 is a rollback, and the endpoint's `model_version` field shows which one is live.

Why it matters

A team retrained a topic classifier and promoted it because its validation accuracy was 2 points higher than the old model's. Both scores came from different validation sets. On the next month's labelled answers the new model was 3 points worse. With the old image deleted, rolling back took a day of rebuilding. Comparing on the same recent data, and keeping the old image tag, would have avoided both problems.

Worked example: weak code

```
❶ if challenger_val_accuracy > champion_val_accuracy:
❷     client.delete_model_version("survey_topics", champion_version)
3     client.set_registered_model_alias("survey_topics", "champion",
    challenger_version)
```

1. **Line 1.** Two scores from different data, and a win of any size counts.
2. **Line 2.** Deletes the only rollback target.

Worked example: strong code

```
1 from statistics import mean
2
3
4 def should_promote(champion: list[float], challenger: list[float],
    margin: float = 0.01) -> bool:
5     """Promote only if the challenger beats the champion on the same
    batches.
6
```

```
    Args:
        champion: The live model's score on each recent labelled batch.
        challenger: The new model's score on the same batches, in the same order.
        margin: Smallest mean gain worth a change.

    Returns:
        True if the mean gain is at least margin and the challenger wins most batches.

    Raises:
        ValueError: If the lists are empty or differ in length.
    """
    if not champion or len(champion) != len(challenger):
        raise ValueError("need the same, non-empty set of batches for both models")
    gain = mean(challenger) - mean(champion)
    wins = sum(new > old for old, new in zip(champion, challenger))
    return gain >= margin and wins > len(champion) / 2
```

1. **Line 9.** Paired scores on the same data: the only fair comparison.
2. **Line 20.** The gain must clear a margin. Being positive is not enough.
3. **Line 21.** A win on one lucky batch does not count; it must win most of them.

Compare champion and challenger on the same recent data, shadow before switching, promote by moving an alias and deploying a new revision, and keep old versions, images and revisions for rollback.

Recap card

- **Champion and challenger** Compare on the same recent labelled batches. Promote only for a gain above a margin, won on most batches.
- **Shadow mode** Serve the champion, run the challenger on the same requests, log both, and review agreement and swaps.
- **Promotion** Move the alias, export, build an image tagged with the version, deploy a new revision.
- **Rollback** Move the alias back and return traffic to the old revision, or redeploy the old image tag. Keep old versions, tags and revisions.

Practice

Exercise 1. Multiple choice (level 1)

A challenger scored 0.91 on its own validation set. The champion scored 0.89 on its own, older validation set. What do you do?

- a) Score both on the same recent labelled data before deciding
- b) Promote the challenger: 0.91 is higher

- c) Keep the champion: older models are safer
- d) Average the two scores and promote if the average rises

Why?

- r1) Scores from different data cannot rank two models: the gap may come from the data, not the models.
- r2) Validation sets always overstate accuracy by 2 points.

Exercise 2. Predict the output (level 2)

In shadow mode, both models labelled the same five answers. What does this print?

```
champion = ["price", "delivery", "staff", "delivery", "price"]
challenger = ["price", "delivery", "delivery", "delivery", "staff"]
swaps = {}
for old, new in zip(champion, challenger):
    if old != new:
        swaps[(old, new)] = swaps.get((old, new), 0) + 1
agreement = sum(old == new for old, new in zip(champion, challenger)) /
  len(champion)
print(agreement, swaps)
```

Exercise 3. Put in order (level 2)

Order the steps of replacing a live model.

- If the signals turn bad, move the alias back and return traffic to the old revision
- Score champion and challenger on the same recent labelled batches
- Run the challenger in shadow on live requests and review the swaps
- Move the champion alias, export, build an image tagged with the version
- Train the challenger and register it as a new version
- Deploy a new revision and watch errors, latency and drift

Exercise 4. Write the function (level 3)

Write `should_promote(champion, challenger, margin=0.01) -> bool`. The lists hold each model's score on the same recent labelled batches, in the same order.

- Promote when the challenger's mean is at least `margin` above the champion's, and the challenger scores higher on more than half the batches.
- Raise `ValueError` if the lists are empty or differ in length.

What you need to know.

`statistics.mean` averages a list, and `zip` pairs two lists position by position:

```
from statistics import mean
mean([2.0, 4.0])                                  # 3.0
list(zip([1, 2], [10, 20]))                       # [(1, 10), (2, 20)]
```

Your code must pass these checks:

- A gain above the margin on most batches is a promotion.
- A big win on one batch is not enough: the challenger must win most batches.
- The mean gain must reach the margin.
- Raise ValueError for empty or unequal lists.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Promote only if the challenger beats the champion on the same batches.
def should_promote(
    champion: list[float],
    challenger: list[float],
    margin: float = 0.01,
) -> bool: ...
```

Starter code:

```
def should_promote(champion, challenger, margin=0.01):
    return max(challenger) > max(champion)
```

Exercise 5. Write the function (level 3)

Write `shadow_report(champion, challenger) -> dict` from the two models' labels on the same requests.

- `agreement`: the share of positions where the labels match, rounded to 4 places.
- `top_swap`: the most frequent `(champion_label, challenger_label)` pair among disagreements, as a tuple, or `None` if they always agree. Break ties by the smallest pair in sorted order.
- Raise `ValueError` if the lists are empty or differ in length.

Your code must pass these checks:

- Agreement is the matching share; top_swap is the commonest (champion, challenger) pair.
- Break ties by the smallest pair in sorted order.
- With no disagreements, top_swap is None.
- Raise ValueError for empty or unequal lists.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Agreement rate and the commonest label swap between two models.
def shadow_report(
    champion: list[str],
    challenger: list[str],
) -> dict[str, object]: ...
```

Starter code:

```python
def shadow_report(champion, challenger):
    return {"agreement": 1.0, "top_swap": None}
```

Exercise 6. Architecture design (level 4)

Plan for rolling back the survey classifier within five minutes if a new version misbehaves.

Options: Keep them, and log every alias move with who, when and why, Delete them after each promotion to keep the registry tidy, Overwrite version 1 with each new model.

1. Old model versions
2. Container images
3. Container Apps revisions

Practical: Register, serve, containerise and monitor a model

Platform: Laptop. Work in `labs/serving/` in this repository. You register model versions in MLflow and serve the champion through an alias, write a FastAPI service with a health route and a validated prediction route, package it as a Docker image, and measure drift with PSI. You need Python 3.12, Docker and the lab's dev extras on the laptop. Deploying to Azure is optional and needs a subscription. The running service and its README are the roadmap's evidence for month 18: one deployed model.

1. Run `cd labs/serving`, create a virtual environment, then `pip install -e ".[dev]"`. Run `pytest`: most tests fail with NotImplementedError. Read `README.md` and every file in `src/serve_lab/`.
2. Write `psi` and `drift_status` in `drift.py` (lesson B27-L5). Run `pytest tests/test_drift_and_model.py`.
3. Write `register`, `promote` and `load_alias` in `registry.py` (lesson B27-L1), then `create_app` in `api.py` (lesson B27-L2). Run `pytest`: every test passes.
4. Set `MLFLOW_TRACKING_URI=sqlite:///mlflow.db`. Run `python -m serve_lab train` twice, once with `--c 0.01`, then `promote 1`, `export` and `serve`. Open `http://127.0.0.1:8000/docs` and send a request.

5. Build and run the image (lesson B27-L3): `docker build -t survey-topics:1 .`, then `docker run --rm -p 8000:8000 survey-topics:1`. Call `/health` and `/predict` and read the JSON log line.
6. Run `python -m serve_lab drift` and compare with the README's reference table. Then promote version 2, export, rebuild as `survey-topics:2`, and roll back to version 1 (lesson B27-L6).
7. Optional: deploy with `az containerapp up` (lesson B27-L4), call the public URL, then delete the resource group. Write your README: the registry steps, the API contract, the image, the drift report and the rollback. Push it to a new GitHub repository.

Sources

- Azure Container Apps revisions: single revision mode is the default, moves traffic once the new revision is ready and deprovisions old revisions; multiple revision mode keeps several active and splits traffic by percentage; deployment labels mode is in preview; up to 100 inactive revisions are kept. (search-only: Microsoft Learn, 'Update and deploy changes in Azure Container Apps', ms.date 2025-10-27, read November 2026.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B27-L1 The model registry, versions and aliases

B27-L1 exercise 1

Hints. (1) What do you change to promote a new model? (2) What do you change to roll back? (3) Neither should touch the serving code.

Answer. a) Promotion and rollback become moving the alias, with no change to serving code or settings Because: r1) The alias separates which model is live from the code that serves it.

With an alias, the registry decides which version is live. Promotion and rollback are one registry call each, and the deployed code stays the same.

Common wrong answers.

- `b`: Both resolve to the same files. The benefit is control, not speed.
- `c`: Aliases are names. Versions keep their files until you delete them.
- `d`: models:/survey_topics/4 loads version 4. The lab's load_alias does exactly that after resolving the alias.
- `a|r2`: Right answer, wrong reason. Caching has nothing to do with it. The alias decouples the live choice from the code.

B27-L1 exercise 2

Hints. (1) Each alias holds one version. (2) Follow champion through three moves. (3) Which versions still carry an alias?

Answer.

```
1 {'champion': 1, 'challenger': 2} [1, 2]
```

`champion` moved from 1 to 2 and back to 1, a promotion followed by a rollback. `challenger` still names version 2. Version 2 is unchanged and can be promoted again.

Common wrong answers.

- `{'champion': 2, 'challenger': 2} [2]`: The last call moved champion back to version 1: that is the rollback.
- `{'champion': 1, 'challenger': 2, 'champion': 2} [1, 2]`: An alias names one version at a time. Moving it replaces the old target.

B27-L1 exercise 3

Hints. (1) The alias name goes in the first call. (2) Which character introduces an alias in a model URI? (3) `champion` and `@`.

Answer.

```
client.set_registered_model_alias("survey_topics", "champion", "3")
model = mlflow.pyfunc.load_model("models:/survey_topics@champion")
```

`set_registered_model_alias(name, alias, version)` moves the alias, and `models:/name@alias` loads whatever it points to.

Common wrong answers.

- g1 = `champion`; g2 = `/`: models:/name/... takes a version number. An alias follows @.

B27-L1 exercise 4

Hints. (1) `(alias is None) == (version is None)` is true when both or neither are given. (2) Check `version < 1` before building the version URI. (3) @ for an alias, / for a version.

Answer.

```
def model_uri(name: str, alias: str | None = None, version: int | None = None) -> str:
    """A registry URI for a model by alias or by version.

    Args:
        name: Registered model name.
        alias: Alias such as "champion".
        version: Version number, from 1.

    Returns:
        "models:/<name>@<alias>" or "models:/<name>/<version>".

    Raises:
        ValueError: Unless exactly one of alias and version is given, or if version is below 1.
    """
    if (alias is None) == (version is None):
        raise ValueError("give exactly one of alias and version")
    if version is not None:
        if version < 1:
            raise ValueError(f"versions start at 1, got {version}")
        return f"models:/{name}/{version}"
    return f"models:/{name}@{alias}"
```

Comparing the two `is None` tests catches both and neither in one line. Testing with `is None` matters: version 0 is falsy and would otherwise look like no version at all.

B27-L1 exercise 5

Hints. (1) Build the alias's own history of versions. (2) Append only when the version differs from the last one. (3) The answer is the second-to-last entry, if there is one.

Answer.

```python
def rollback_target(moves: list[tuple[str, int]], alias: str =
    "champion") -> int | None:
    """The version an alias pointed to before its current one.

    Args:
        moves: (alias, version) pairs in time order.
        alias: The alias to follow.

    Returns:
        The previous distinct version, or None if there is none.
    """
    history: list[int] = []
    for name, version in moves:
        if name == alias and (not history or history[-1] != version):
            history.append(version)
    return history[-2] if len(history) >= 2 else None
```

Filtering by alias and dropping repeats gives the real sequence of champions. The second-to-last is where a rollback goes.

B27-L1 exercise 6

Hints. (1) How many times is the alias resolved? (2) What if the alias moves between the two calls? (3) Resolve once, then load that version.

Answer. The bug is on line 6.

Line 6 resolves `@champion` once to load the model, and line 7 resolves it again for the version. If the alias moves in between, they disagree. Resolve the version first, then load `models:/{name}/{version}`.

Common wrong answers.

- Line 7: Asking the registry for the alias's version is right. The trouble is that the model was loaded from a separate lookup.
- Line 8: Returning both is right. They must come from the same lookup.

B27-L2 Serving a model with FastAPI

B27-L2 exercise 1

Hints. (1) How often does the endpoint run? (2) How often does start-up run? (3) Loading is the slow part.

Answer. a) It loads once when the server starts, instead of on every request Because: r1) Loading is slow and uses memory; doing it per request multiplies both by the number of requests.

The lifespan code before `yield` runs once at start-up. Every request then uses the loaded model from `app.state`.

Common wrong answers.

- `b`: Endpoints may read files. The cost is doing it again for every request.
- `c`: Schemas validate requests. The lifespan function only runs start-up and shut-down code.
- `d`: The opposite: the lifespan runs when the server starts, not when the module is imported.
- `a|r2`: Right answer, wrong reason. It is about running once, not about a faster thread.

B27-L2 exercise 2

Hints. (1) Two rules: one on the list, one on each item. (2) An empty list breaks the list rule. (3) loc includes the item's position.

Answer.

```
ok
too_short ('texts',)
string_too_short ('texts', 1)
```

The list rule rejects `[]` as `too_short`. The item rule rejects the empty string at index 1 as `string_too_short`, and `loc` points at it. FastAPI returns these same errors in a 422 response.

Common wrong answers.

- `ok too_short ('texts',) ok`: Each item must have at least 1 character, so the empty string at position 1 fails.
- `ok ok string_too_short ('texts', 1)`: The list itself needs at least 1 item, so an empty list fails.

B27-L2 exercise 3

Hints. (1) Which method sends a JSON body? (2) Which status means the service is unavailable for now? (3) `post` and `503`.

Answer.

```python
@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest) -> PredictResponse: ...

@app.get("/health")
```

```
def health() -> dict[str, str]:
    if getattr(app.state, "model", None) is None:
        raise HTTPException(status_code=503, detail="model not loaded")
    return {"status": "ok"}
```

A prediction request carries a body, so it is a POST. 503 Service Unavailable tells a load balancer or health probe to wait and try again.

Common wrong answers.

- g1 = `get`; g2 = `503`: Predictions take a JSON body, which needs POST.
- g1 = `post`; g2 = `500`: 500 says the server failed. 503 says it is not ready yet, which tells the platform to wait.

B27-L2 exercise 4

Hints. (1) Check that texts is a non-empty list first. (2) Then check every item is a non-empty string. (3) Only then compare the length with max_batch, using `>`.

Answer.

```
def request_status(texts: object, max_batch: int) -> int:
    """HTTP status for a request's texts value, before any prediction.

    Args:
        texts: The "texts" value from the request body.
        max_batch: Most texts one request may hold.

    Returns:
        422 for invalid input, 413 for too many texts, else 200.
    """
    if not isinstance(texts, list) or not texts:
        return 422
    if not all(isinstance(t, str) and t for t in texts):
        return 422
    if len(texts) > max_batch:
        return 413
    return 200
```

Validation comes before the size limit, as in FastAPI: a malformed body is 422 whatever its size. A bare string is iterable, so without the `isinstance` check `"late"` would pass as four one-letter texts.

B27-L2 exercise 5

Hints. (1) Check `probs.shape[1] == len(classes)`. (2) `probs.argmax(axis=1)` gives the best column per row. (3) `round(float(row[i]), 4)`.

Answer.

```
import numpy as np

```

```
3
4  def to_predictions(probs: np.ndarray, classes: list[str]) ->
       list[dict[str, str | float]]:
5      """Label and confidence for each row of class probabilities.
6
7      Args:
8          probs: Shape (n_texts, n_classes).
9          classes: Class names in column order.
10
11     Returns:
12         One {"label", "confidence"} dict per row.
13
14     Raises:
15         ValueError: If the number of columns differs from len(classes).
16     """
17     if probs.ndim != 2 or probs.shape[1] != len(classes):
18         raise ValueError(f"probs has shape {probs.shape} but there are
       {len(classes)} classes")
19     best = probs.argmax(axis=1)
20     return [
21         {"label": classes[int(i)], "confidence": round(float(row[i]), 4)}
22         for i, row in zip(best, probs)
23     ]
```

`argmax(axis=1)` picks a column per row. `float(...)` turns a NumPy number into a plain float, which JSON and pydantic handle directly, and rounding keeps responses short.

B27-L2 exercise 6

Hints. (1) What runs on every request? (2) Which step only needs to happen once? (3) Move it into a lifespan function.

Answer. The bug is on line 9.

Line 9 reads the model from disk for every request. Load it once in a lifespan function and keep it on `app.state`.

Common wrong answers.

- Line 10: Predicting is the part that has to run per request. The time goes before it.
- Line 7: The route and its schemas are right. Look at what runs inside the function.

B27-L3 Packaging a model service as a container image

B27-L3 exercise 1

Hints. (1) Which address does the log show? (2) Who can reach a loopback address? (3) --host 0.0.0.0.

Answer. a) 127.0.0.1 inside the container accepts connections only from inside the container. Start uvicorn with --host 0.0.0.0 Because: r1) The published port forwards to the container's network interface, where nothing is listening.

Uvicorn's default host is 127.0.0.1, the container's own loopback. Docker forwards the published port to the container's network interface, so uvicorn must listen on 0.0.0.0.

Common wrong answers.

- `b`: EXPOSE documents the port. It does not change where uvicorn listens.
- `c`: 8000 is an ordinary port. The bind address is the problem.
- `d`: A loading model would answer /health with 503. Here nothing answers at all.
- `a|r2`: Right answer, wrong reason. EXPOSE does not open anything. Traffic arrives on the container's interface, and nothing listens there.

B27-L3 exercise 2

Hints. (1) Which strings are only digits? (2) Is 0 in the range? (3) A leading space is not a digit.

Answer.

```
100
rejected '0'
rejected '50.5'
rejected ' 20'
```

Only "100" is all digits and in range. Rejecting bad settings at start-up stops the container before it serves anything, with a message naming the variable.

Common wrong answers.

- `100 rejected '0' 50 20`: isdigit is False for '50.5' and for ' 20', because '.' and the space are not digits.
- `100 0 rejected '50.5' rejected ' 20'`: 0 is a digit string, but it fails the range check of 1 to 1000.

B27-L3 exercise 3

Hints. (1) What changes least often? (2) Install packages while still root. (3) CMD comes last.

Answer.

1. FROM python:3.12-slim
2. COPY requirements-serve.txt .
3. RUN pip install --no-cache-dir -r requirements-serve.txt
4. COPY src/ src/ and COPY model/ model/
5. RUN useradd --create-home appuser, then USER appuser
6. CMD ["uvicorn", ..., "--host", "0.0.0.0"]

Base image, requirements, install, then code and model, then a non-root user and the start command. Docker reuses every step before the first changed input.

Common wrong answers.

- Copying code first means every code change invalidates the install step, so all packages reinstall.
- Switching user first makes pip install as a user without rights to the system site-packages. Install as root, then switch.

B27-L3 exercise 4

Hints. (1) Strip each line first. (2) `line.partition("==")` returns name, separator and version. (3) An empty separator, name or version means the line is not a pin.

Answer.

```python
def parse_requirements(lines: list[str]) -> dict[str, str]:
    """Exact pins from a requirements file.

    Args:
        lines: The file's lines.

    Returns:
        Package name to pinned version.

    Raises:
        ValueError: If a line is not of the form name==version.
    """
    pins: dict[str, str] = {}
    for raw in lines:
        line = raw.strip()
        if not line or line.startswith("#"):
            continue
```

```
        name, sep, version = line.partition("==")
        if not sep or not name or not version:
            raise ValueError(f"not pinned with ==: {line!r}")
        pins[name] = version
    return pins
```

A check like this in CI keeps the serving image reproducible: any range or bare name fails the build before it reaches production.

B27-L3 exercise 5

Hints. (1) `env.get(name, default)` supplies the defaults. (2) Check `raw.isdigit()` before `int(raw)`. (3) Then check the range and name MAX_BATCH in the message.

Answer.

```
def load_settings(env: dict[str, str]) -> dict[str, str | int]:
    """Service settings from environment variables, checked at start-up.

    Args:
        env: Environment variables, such as dict(os.environ).

    Returns:
        {"model_dir": str, "max_batch": int}.

    Raises:
        ValueError: If MODEL_DIR is empty or MAX_BATCH is not a whole number from 1 to 1000.
    """
    model_dir = env.get("MODEL_DIR", "model")
    if not model_dir:
        raise ValueError("MODEL_DIR must not be empty")
    raw = env.get("MAX_BATCH", "100")
    if not raw.isdigit() or not 1 <= int(raw) <= 1_000:
        raise ValueError(f"MAX_BATCH must be a whole number from 1 to 1000, got {raw!r}")
    return {"model_dir": model_dir, "max_batch": int(raw)}
```

Checking settings when the app starts turns a typo in the deployment into a clear start-up failure instead of odd behaviour under load.

B27-L3 exercise 6

Hints. (1) Docker reuses a step until its inputs change. (2) Which step's inputs include your code? (3) Copy the requirements file alone first.

Answer. The bug is on line 3.

`COPY . .` puts every source file into the step before `pip install`, so any code change invalidates the install. Copy `requirements-serve.txt` first, install, then copy the code.

Common wrong answers.

- Line 4: The install itself is right. It reruns because something before it changed.
- Line 1: The slim base image is a good choice. Look at what is copied before the install.

B27-L4 Deploying the container to Azure Container Apps

B27-L4 exercise 1

Hints. (1) What does a TCP probe check? (2) When does uvicorn open the port? (3) When does the model finish loading?

Answer. a) The default readiness probe is a TCP check, which passes when the port opens, before the model has loaded Because: r1) A TCP probe only checks that something listens. An HTTP probe on /health checks that the model is ready.

uvicorn listens before the lifespan has finished loading the model, so a TCP probe passes too early. An HTTP readiness probe on `/health`, which answers 503 until the model is ready, holds traffic back.

Common wrong answers.

- `b`: Liveness restarts unhealthy replicas. Readiness decides when traffic arrives.
- `c`: A TCP probe never sees an HTTP status. It only opens a connection.
- `d`: In single revision mode, traffic moves to the new revision once its readiness probes pass.
- `a|r2`: Right answer, wrong reason. Probes run every few seconds. The problem is what a TCP probe checks.

B27-L4 exercise 2

Hints. (1) ceil rounds up. (2) The maximum is 10. (3) The last call sets a minimum of 1.

Answer.

```
1 [0, 1, 4, 10] 1
```

7 concurrent requests round up to 1 replica, 35 to 4, and 250 would need 25 but stop at 10. With a minimum of 1, an idle app keeps one warm replica.

Common wrong answers.

- `[0, 0, 3, 25] 0`: ceil rounds up, so 7 requests need 1 replica and 35 need 4. The maximum caps 250 at 10.
- `[1, 1, 4, 10] 1`: The default minimum is 0, so no traffic means no replicas.

B27-L4 exercise 3

Hints. (1) Who should reach the app? (2) Which port does the CMD give uvicorn? (3) `external` and `8000`.

Answer.

```
az containerapp up --name survey-topics --source . --ingress external --target-port 8000
```

`--ingress external` gives the app a public HTTPS address, and `--target-port 8000` sends that traffic to uvicorn's port inside the container.

Common wrong answers.

- g1 = `internal`; g2 = `8000`: internal ingress is reachable only from inside the Container Apps environment.
- g1 = `external`; g2 = `80`: The target port is the port the container listens on: uvicorn uses 8000.

B27-L4 exercise 4

Hints. (1) Check the overlap with a set intersection. (2) Secret items are `f"{name}=secretref:{secret}"`. (3) Sort by variable name at the end.

Answer.

```
def env_var_args(plain: dict[str, str], secret_refs: dict[str, str]) -> list[str]:
    """Items for az containerapp --env-vars, secrets as secretref: references.

    Args:
        plain: Variable name to value.
        secret_refs: Variable name to the app secret it reads.

    Returns:
        NAME=value and NAME=secretref:secret items, sorted by name.

    Raises:
        ValueError: If a name is empty or appears in both mappings.
    """
    clash = set(plain) & set(secret_refs)
    if clash:
        raise ValueError(f"both plain and secret: {sorted(clash)}")
```

```
    if "" in plain or "" in secret_refs:
        raise ValueError("variable names must not be empty")
    items = {name: f"{name}={value}" for name, value in plain.items()}
    items |= {name: f"{name}=secretref:{secret}" for name, secret in secret_refs.items()}
    return [items[name] for name in sorted(items)]
```

`secretref:` makes Container Apps fill the variable from the app's secret at run time, so the value never appears in the command. A name in both mappings is a mistake to stop, not to resolve silently.

B27-L4 exercise 5

Hints. (1) Check each character: a dash, or a lower-case ASCII letter or digit. (2) Then the first and last characters, and `"--" in suffix`. (3) Length at most 64.

Answer.

```
def revision_name(app: str, suffix: str) -> str:
    """A revision name, after checking the suffix rules.

    Args:
        app: Container app name.
        suffix: Revision suffix.

    Returns:
        "<app>-<suffix>".

    Raises:
        ValueError: If the suffix breaks a naming rule.
    """
    allowed = all(ch == "-" or (ch.isalnum() and ch.isascii() and not ch.isupper()) for ch in suffix)
    if (
        not suffix
        or len(suffix) > 64
        or not allowed
        or not suffix[0].isalpha()
        or not suffix[-1].isalnum()
        or "--" in suffix
    ):
        raise ValueError(f"invalid revision suffix: {suffix!r}")
    return f"{app}-{suffix}"
```

Checking the suffix before calling Azure gives a clear error in your own deploy script, instead of a failed deployment.

B27-L4 exercise 6

Hints. (1) Compare the load time with the caller's timeout. (2) Where can a key live outside the image and the command? (3) Which probe knows when the model is ready?

Answer.

- One warm replica: a cold start of over 15 seconds would exceed the 10-second timeout.
- The key stays in Key Vault and reaches the container as a secretref variable.
- An HTTP readiness probe on /health keeps traffic away until the model has loaded.

Each choice follows from one fact: the model needs 15 seconds, the caller waits 10, and the key must stay out of code and commands.

Common wrong answers.

- replicas = `zero`: After an idle spell the first caller waits for a start plus a 15-second load, past the 10-second timeout.
- replicas = `ten`: Ten replicas around the clock costs far more than the timeout needs. One warm replica removes the cold start.
- secret = `plain`: A plain value shows in the command, the shell history and the app's settings.
- secret = `image`: Anything in the image is readable by anyone who can pull it, and rotating the key means rebuilding.
- probe = `tcp`: TCP passes as soon as uvicorn listens, 15 seconds before the model is ready.
- probe = `none`: Without readiness, traffic can reach a replica whose model is still loading.

B27-L5 Monitoring a served model for drift

B27-L5 exercise 1

Hints. (1) What do you see on every request? (2) Which signals need no true label? (3) Inputs, outputs, service.

Answer. a) Input feature distributions, predicted class shares, confidence, error rate and latency Because: r1) These need no labels and change as soon as the incoming data changes.

Drift in inputs or outputs is visible at once and often precedes a fall in accuracy. It tells you where to look while labels are on their way.

Common wrong answers.

- `b`: Inputs, outputs and the service itself are all visible now and often move before accuracy does.
- `c`: The test set does not change, so its accuracy cannot reveal drift in new data.
- `d`: Training loss describes the past training run, not today's traffic.
- `a|r2`: Right answer, wrong reason. They are early signs, not accuracy. Confirm with labels when they arrive.

B27-L5 exercise 2

Hints. (1) First term: (0.25 - 0.5) * ln(0.5). (2) Both terms are positive. (3) 0.1733 + 0.1014.

Answer.

```
0.2747
```

Each term `(a - e) * ln(a / e)` is positive whenever a and e differ, because both factors share a sign. 0.2747 sits above the common 0.25 alert line.

Common wrong answers.

- `0.0`: The shares differ, so each term is positive. PSI is 0 only when they match.
- `0.5`: That is the sum of absolute differences. PSI weights each difference by the log of the ratio.

B27-L5 exercise 3

Hints. (1) Both shares need the same floor. (2) PSI uses the natural logarithm of the ratio. (3) `EPS` and `log`.

Answer.

```
e, a = np.clip(e, EPS, None), np.clip(a, EPS, None)
return float(np.sum((a - e) * np.log(a / e)))
```

Raising every share to at least `EPS` keeps the ratio finite. `np.log` is the natural logarithm the formula uses.

Common wrong answers.

- g1 = `0`; g2 = `log`: A floor of 0 leaves empty bins at 0, and log(0 / e) is minus infinity.

B27-L5 exercise 4

Hints. (1) Edges come from expected only. (2) Shares: bincount of the bin numbers, divided by the array's length. (3) `np.clip(share, 1e-4, None)` before the log.

Answer.

```python
import numpy as np

EPS = 1e-4


def psi(expected: np.ndarray, actual: np.ndarray, bins: int = 10) -> float:
    """Population stability index of actual against expected.

    Args:
        expected: Reference values.
        actual: Values to check.
        bins: Number of quantile bins, at least 2.

    Returns:
        The PSI, 0 when the shares match.

    Raises:
        ValueError: If bins is below 2 or either array is empty.
    """
    if bins < 2:
        raise ValueError(f"bins must be at least 2, got {bins}")
    if len(expected) == 0 or len(actual) == 0:
        raise ValueError("expected and actual must not be empty")
    inner = np.unique(np.quantile(expected, np.linspace(0, 1, bins + 1))[1:-1])
    n_bins = len(inner) + 1
    e = np.bincount(np.searchsorted(inner, expected, side="right"), minlength=n_bins) / len(expected)
    a = np.bincount(np.searchsorted(inner, actual, side="right"), minlength=n_bins) / len(actual)
    e, a = np.clip(e, EPS, None), np.clip(a, EPS, None)
    return float(np.sum((a - e) * np.log(a / e)))
```

Both arrays must share the reference's bins, or each would look evenly spread and PSI would stay near 0. The floor keeps empty bins finite.

B27-L5 exercise 5

Hints. (1) Guard the empty cases before dividing. (2) Weighted mean: sum of confidence times n_texts, over total texts. (3) `np.percentile(latencies, 95)`.

Answer.

```python
import numpy as np


def summarise(records: list[dict[str, float]]) -> dict[str, float]:
    """Request count, text count, p95 latency and text-weighted mean confidence.

    Args:
        records: One prediction log record per request.

    Returns:
        {"requests", "texts", "p95_latency_ms", "mean_confidence"}.

    Raises:
        ValueError: If there are no records or no texts.
    """
    if not records:
        raise ValueError("no records")
    texts = sum(int(r["n_texts"]) for r in records)
    if texts == 0:
        raise ValueError("no texts in the records")
    weighted = sum(r["mean_confidence"] * r["n_texts"] for r in records) / texts
    p95 = float(np.percentile([r["latency_ms"] for r in records], 95))
    return {
        "requests": len(records),
        "texts": texts,
        "p95_latency_ms": round(p95, 2),
        "mean_confidence": round(weighted, 4),
    }
```

A request with 50 texts says more about confidence than a request with 1, so weight by texts. The 95th percentile shows the slow tail that a mean hides.

B27-L5 exercise 6

Hints. (1) Where do actual's bin edges come from? (2) On its own quantiles, how are actual's values spread across bins? (3) Use expected's edges for both.

Answer. The bug is on line 7, 9.

Fixed version:

```python
import numpy as np


def psi(expected, actual, bins=10):
    q = np.linspace(0, 1, bins + 1)
    e_inner = np.quantile(expected, q)[1:-1]
    e = np.bincount(np.searchsorted(e_inner, expected, side="right"), minlength=bins) / len(expected)
```

```
 8     a = np.bincount(np.searchsorted(e_inner, actual, side="right"),
     minlength=bins) / len(actual)
 9     e, a = np.clip(e, 1e-4, None), np.clip(a, 1e-4, None)
10     return float(np.sum((a - e) * np.log(a / e)))
```

Lines 7 and 9 bin `actual` on its own quantiles, so each bin holds about a tenth of it, whatever its shape. Both arrays must use the reference's edges for the shares to show a shift.

Common wrong answers.

- Line 10: The floor is right. The shares were already wrong before it.
- Line 6, 8: Binning expected on its own quantiles is right. The current data must use the same edges.

B27-L6 Retraining, promotion and rollback

B27-L6 exercise 1

Hints. (1) Where did each score come from? (2) What else differs besides the model? (3) Same data, then compare.

Answer. a) Score both on the same recent labelled data before deciding Because: r1) Scores from different data cannot rank two models: the gap may come from the data, not the models.

A fair comparison holds the data fixed. Score both models on the same recent labelled batches, then decide.

Common wrong answers.

- `b`: The two numbers come from different data, so the 2-point gap may say nothing about the models.
- `c`: Age is not evidence. Measure both on the same data.
- `d`: Averaging scores from different data still compares unlike things.
- `a|r2`: Right answer, wrong reason. There is no fixed overstatement. The data differ, so the scores cannot be compared.

B27-L6 exercise 2

Hints. (1) Count matching positions. (2) Positions 2 and 4 differ. (3) Keys are (old, new).

Answer.

```
1 0.6 {('staff', 'delivery'): 1, ('price', 'staff'): 1}
```

The models agree on 3 of 5 answers. The swaps show where the challenger would change users' results: one staff answer becomes delivery, one price answer becomes staff.

Common wrong answers.

- `0.4 {('staff', 'delivery'): 1, ('price', 'staff'): 1}`: Three of five labels match: positions 0, 1 and 3.
- `0.6 {('delivery', 'staff'): 1, ('staff', 'price'): 1}`: Each key is (champion, challenger): the old label first.

B27-L6 exercise 3

Hints. (1) Evidence before the switch. (2) Users should see nothing until promotion. (3) Rollback is the safety net after deployment.

Answer.

1. Train the challenger and register it as a new version
2. Score champion and challenger on the same recent labelled batches
3. Run the challenger in shadow on live requests and review the swaps
4. Move the champion alias, export, build an image tagged with the version
5. Deploy a new revision and watch errors, latency and drift
6. If the signals turn bad, move the alias back and return traffic to the old revision

Register, compare offline, shadow online, promote, deploy and watch. Rollback reverses the last two steps, which only works if old versions and revisions still exist.

Common wrong answers.

- Promotion comes after the evidence: comparison and shadow first.
- Shadow mode runs before the switch, while users still get the champion's answers.

B27-L6 exercise 4

Hints. (1) Guard the lists first. (2) Mean gain: `mean(challenger) - mean(champion)`. (3) Count batches where the challenger is higher, and compare with half.

Answer.

```
1 from statistics import mean
2
3
```

```python
def should_promote(champion: list[float], challenger: list[float], margin: float = 0.01) -> bool:
    """Promote only if the challenger beats the champion on the same batches.

    Args:
        champion: Live model's score per batch.
        challenger: New model's score on the same batches.
        margin: Smallest mean gain worth a change.

    Returns:
        True if the mean gain reaches margin and the challenger wins most batches.

    Raises:
        ValueError: If the lists are empty or differ in length.
    """
    if not champion or len(champion) != len(challenger):
        raise ValueError("need the same, non-empty set of batches for both models")
    gain = mean(challenger) - mean(champion)
    wins = sum(new > old for old, new in zip(champion, challenger))
    return gain >= margin and wins > len(champion) / 2
```

The mean alone can be carried by one lucky batch. Requiring wins on most batches as well guards against that, and the margin keeps you from swapping models for noise.

B27-L6 exercise 5

Hints. (1) Count disagreements as (old, new) pairs in a dict. (2) Agreement is 1 minus disagreements over the total. (3) `min(swaps, key=lambda p: (-swaps[p], p))` picks the commonest, then the smallest.

Answer.

```python
def shadow_report(champion: list[str], challenger: list[str]) -> dict[str, object]:
    """Agreement rate and the commonest label swap between two models.

    Args:
        champion: Live model's labels.
        challenger: Shadow model's labels for the same requests.

    Returns:
        {"agreement": float, "top_swap": (old, new) or None}.

    Raises:
        ValueError: If the lists are empty or differ in length.
    """
    if not champion or len(champion) != len(challenger):
        raise ValueError("need two non-empty lists of the same length")
```

```
    swaps: dict[tuple[str, str], int] = {}
    for old, new in zip(champion, challenger):
        if old != new:
            swaps[(old, new)] = swaps.get((old, new), 0) + 1
    agreement = round(1 - sum(swaps.values()) / len(champion), 4)
    top = min(swaps, key=lambda pair: (-swaps[pair], pair)) if swaps else None
    return {"agreement": agreement, "top_swap": top}
```

The top swap tells you which users would notice the change first. A fixed tie-break makes the report the same on every run, so two reports can be compared.

B27-L6 exercise 6

Hints. (1) A rollback needs something to roll back to. (2) Can you name the previous image? (3) Which revision mode keeps the old revision able to serve?

Answer.

- Keep every registered version and an audit trail of alias moves: the rollback target must exist and be known.
- Tag images by model version, so the previous image is ready to deploy.
- Multiple revision mode keeps the old revision able to take traffic again within minutes.

Each layer keeps its previous state: registry versions, image tags and an active old revision. Rollback is then a few commands, not a rebuild.

Common wrong answers.

- registry = `delete`: A deleted version cannot be promoted back.
- registry = `overwrite`: Overwriting destroys the old model and makes version numbers meaningless.
- images = `latest`: latest moves with every build, so the previous image has no name to deploy.
- images = `rebuild`: A rebuild takes time and may not reproduce the old image if dependencies changed.
- revisions = `single`: Single revision mode shuts down the old revision, so a rollback means deploying the old image as a new revision, which must provision and pass its probes first.
- revisions = `delete`: Recreating the app is the slowest possible rollback, and it loses settings.